Name : Moshitha kolanjiyappan

Reg no. 192421205

DAA-LAB-CHAPTER-3

TOPIC 3 : DIVIDE AND CONQUER

1. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

main.c

Run

Output

Clear

```c
#include <stdio.h>

void findMinMax(int a[], int l, int r, int *min,
    int *max)
{
    int min1, max1, min2, max2, mid;

    if (l == r) {
        *min = *max = a[l];
        return;
    }

    mid = (l + r) / 2;

    findMinMax(a, l, mid, &min1, &max1);
    findMinMax(a, mid + 1, r, &min2, &max2);

    *min = (min1 < min2) ? min1 : min2;
    *max = (max1 > max2) ? max1 : max2;
}

int main()
{
    int a[] = {5,7,3,4,9,12,6,2};
    int n = 8, min, max;

    findMinMax(a, 0, n-1, &min, &max);

    printf("Min = %d\nMax = %d", min, max);
    return 0;
}
```

```
Min = 2
Max = 12

=== Code Execution Successful ===
```

2. Consider an array of integers sorted in ascending order: 2,4,6,8,10,12,14,18. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

main.c

Run

Output

Clear

```c
#include <stdio.h>

int main()
{
    int a[] = {2,4,6,8,10,12,14,18};
    int n = 8;

    printf("Min = %d\n", a[0]);
    printf("Max = %d", a[n-1]);

    return 0;
}
```

```
Min = 2
Max = 18

=== Code Execution Successful ===
```

3. You are given an unsorted array 31,23,35,27,11,21,15,28. Write a program for Merge Sort and implement using any programming language of your choice. :

main.c

Run

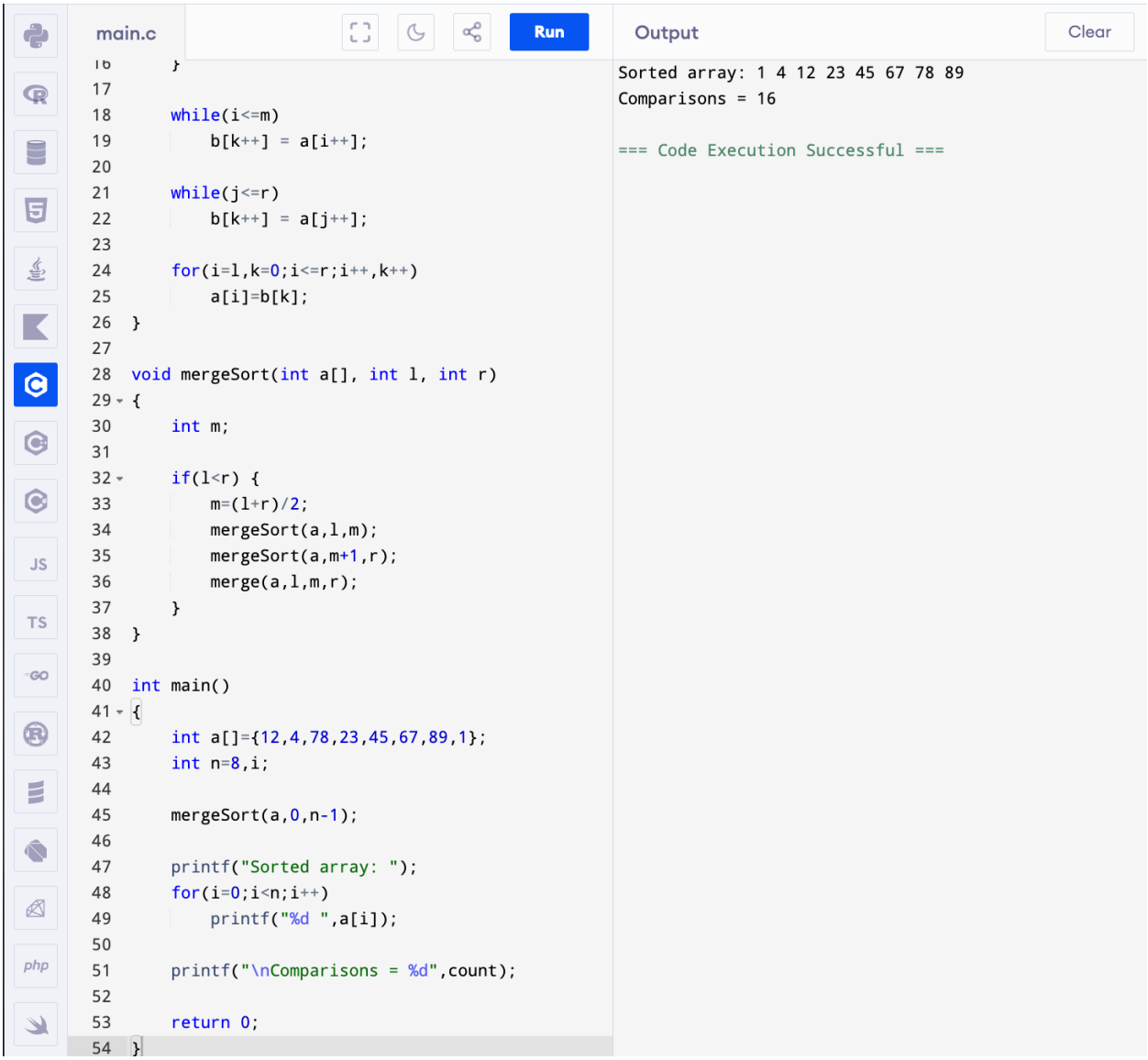
Output

Clear

```c
        else
            b[k++] = a[j++];
    }

    while(i<=m)
        b[k++] = a[i++];

    while(j<=r)
        b[k++] = a[j++];

    for(i=l,k=0; i<=r; i++,k++)
        a[i] = b[k];
}

void mergeSort(int a[], int l, int r)
{
    int m;

    if(l < r) {
        m = (l+r)/2;
        mergeSort(a,l,m);
        mergeSort(a,m+1,r);
        merge(a,l,m,r);
    }
}

int main()
{
    int a[] = {31,23,35,27,11,21,15,28};
    int n=8, i;

    mergeSort(a,0,n-1);

    printf("Sorted array: ");
    for(i=0;i<n;i++)
        printf("%d ",a[i]);

    return 0;
}
```

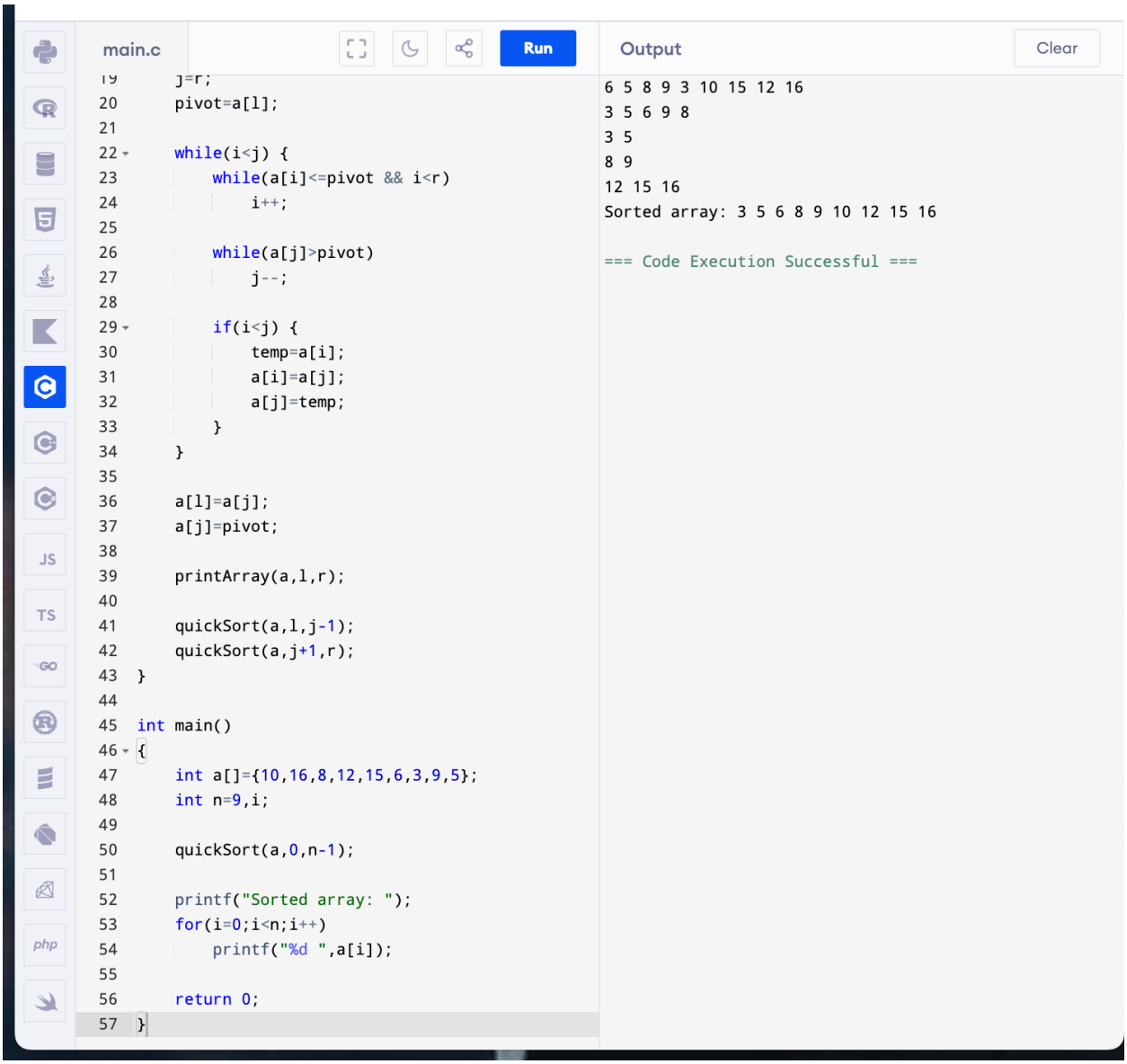
```
Sorted array: 11 15 21 23 27 28 31 35

=== Code Execution Successful ===
```

4. Implement the Merge Sort algorithm in a programming language of your choice and test it on the array 12,4,78,23,45,67,89,1. Modify your implementation to count the number of comparisons made during the sorting process. Print this count along with the sorted array.

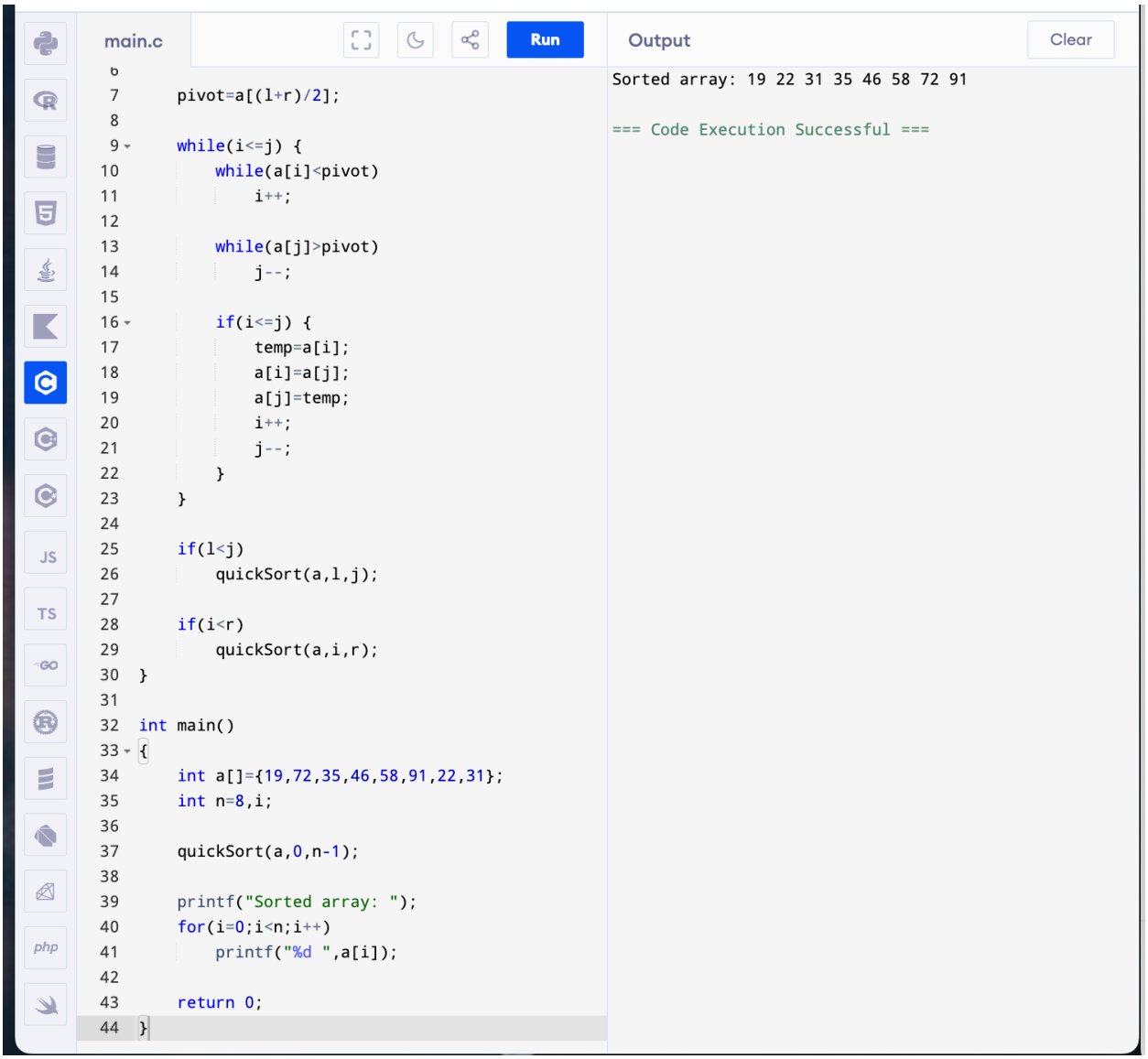



```
    }

    while(i<=m)
        b[k++] = a[i++];

    while(j<=r)
        b[k++] = a[j++];

    for(i=l,k=0;i<=r;i++,k++)
        a[i]=b[k];
}

void mergeSort(int a[], int l, int r)
{
    int m;

    if(l<r) {
        m=(l+r)/2;
        mergeSort(a,l,m);
        mergeSort(a,m+1,r);
        merge(a,l,m,r);
    }
}

int main()
{
    int a[]={12,4,78,23,45,67,89,1};
    int n=8,i;

    mergeSort(a,0,n-1);

    printf("Sorted array: ");
    for(i=0;i<n;i++)
        printf("%d ",a[i]);

    printf("\nComparisons = %d",count);

    return 0;
}
```

5. Given an unsorted array 10,16,8,12,15,6,3,9,5 Write a program to perform Quick Sort. Choose the first element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted.



6. Implement the Quick Sort algorithm in a programming language of your choice and test it on the array 19,72,35,46,58,91,22,31. Choose the middle element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted. Execute your code and show the sorted array.




```c
    pivot=a[(l+r)/2];

    while(i<=j) {
        while(a[i]<pivot)
            i++;

        while(a[j]>pivot)
            j--;

        if(i<=j) {
            temp=a[i];
            a[i]=a[j];
            a[j]=temp;
            i++;
            j--;
        }
    }

    if(l<j)
        quickSort(a,l,j);

    if(i<r)
        quickSort(a,i,r);
}

int main()
{
    int a[]={19,72,35,46,58,91,22,31};
    int n=8,i;

    quickSort(a,0,n-1);

    printf("Sorted array: ");
    for(i=0;i<n;i++)
        printf("%d ",a[i]);

    return 0;
}
```

7. Implement the Binary Search algorithm in a programming language of your choice and test it on the array 5,10,15,20,25,30,35,40,45 to find the position of the element 20. Execute your code and provide the index of the element 20. Modify your implementation to count the number of comparisons made during the search process. Print this count along with the result.

main.c

Run

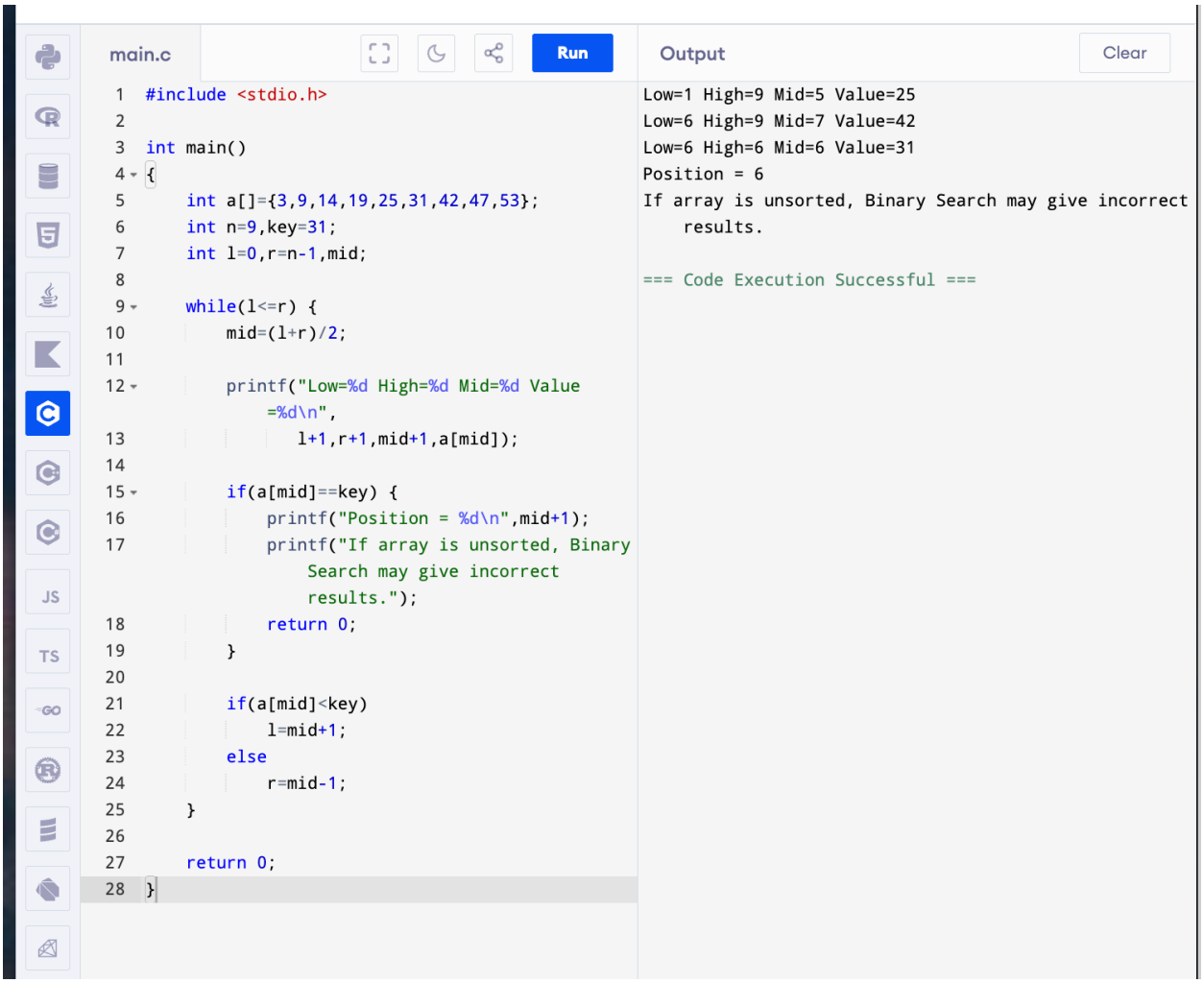
```c
#include <stdio.h>

int main()
{
    int a[]={5,10,15,20,25,30,35,40,45};
    int n=9,key=20;
    int l=0,r=n-1,mid,count=0;

    while(l<=r) {
        mid=(l+r)/2;
        count++;

        if(a[mid]==key) {
            printf("Index = %d\n",mid);
            printf("Position = %d\n",mid+1);
            printf("Comparisons = %d",count);
            return 0;
        }

        if(a[mid]<key)
            l=mid+1;
        else
            r=mid-1;
    }

    printf("Element not found");

    return 0;
}
```

Output

Clear

```
Index = 3
Position = 4
Comparisons = 4

=== Code Execution Successful ===
```

8. You are given a sorted array 3,9,14,19,25,31,42,47,53 and asked to find the position of the element 31 using Binary Search. Show the mid-point calculations and the steps involved in finding the element. Display, what would happen if the array was not sorted, how would this impact the performance and correctness of the Binary Search algorithm?



9. Given an array of points where points[i] = [xi, yi] represents a point on the X-Y plane and an integer k, return the k closest points to the origin (0, 0).

main.c

Run

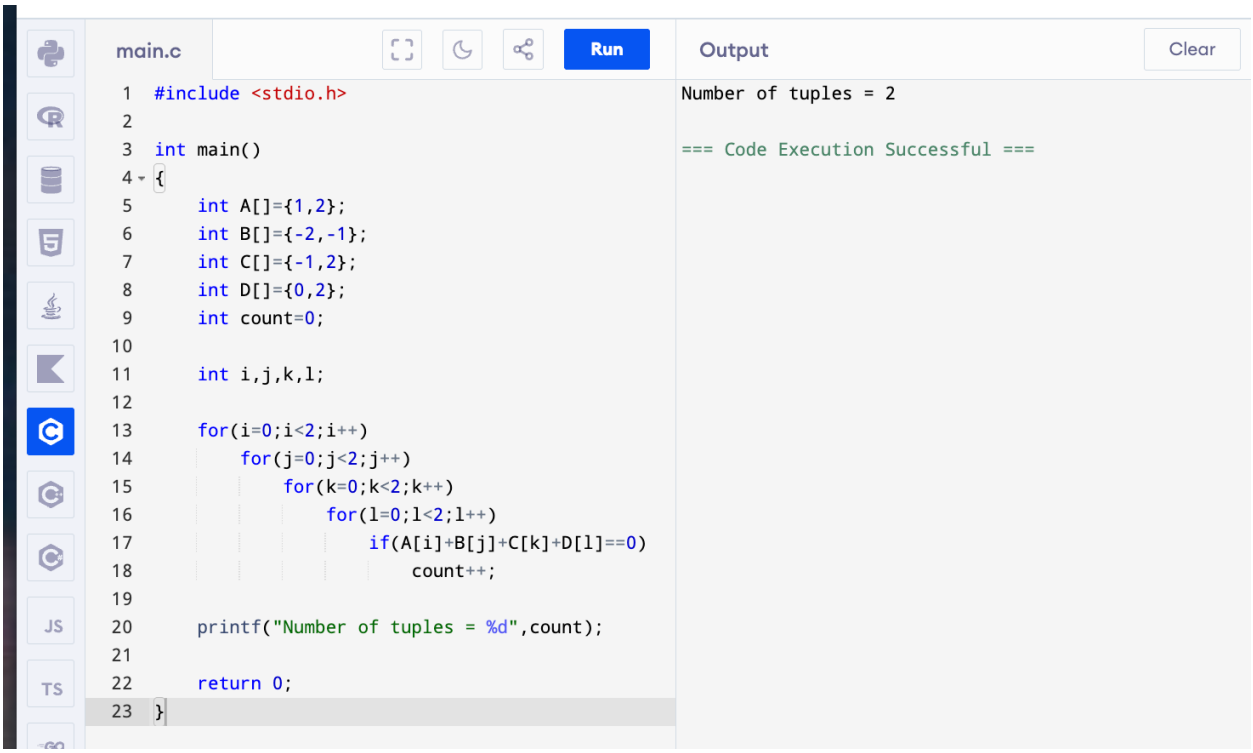
```c
#include <stdio.h>

int main()
{
    int p[4][2]={{1,3},{-2,2},{5,8},{0,1}};
    int d[4],i,j,temp;

    for(i=0;i<4;i++)
        d[i]=p[i][0]*p[i][0]+p[i][1]*p[i][1];

    for(i=0;i<4;i++) {
        for(j=i+1;j<4;j++) {
            if(d[i]>d[j]) {
                temp=d[i];
                d[i]=d[j];
                d[j]=temp;

                temp=p[i][0];
                p[i][0]=p[j][0];
                p[j][0]=temp;

                temp=p[i][1];
                p[i][1]=p[j][1];
                p[j][1]=temp;
            }
        }
    }

    printf("Closest points: ");
    for(i=0;i<2;i++)
        printf("[%d,%d] ",p[i][0],p[i][1]);

    return 0;
}
```

Output

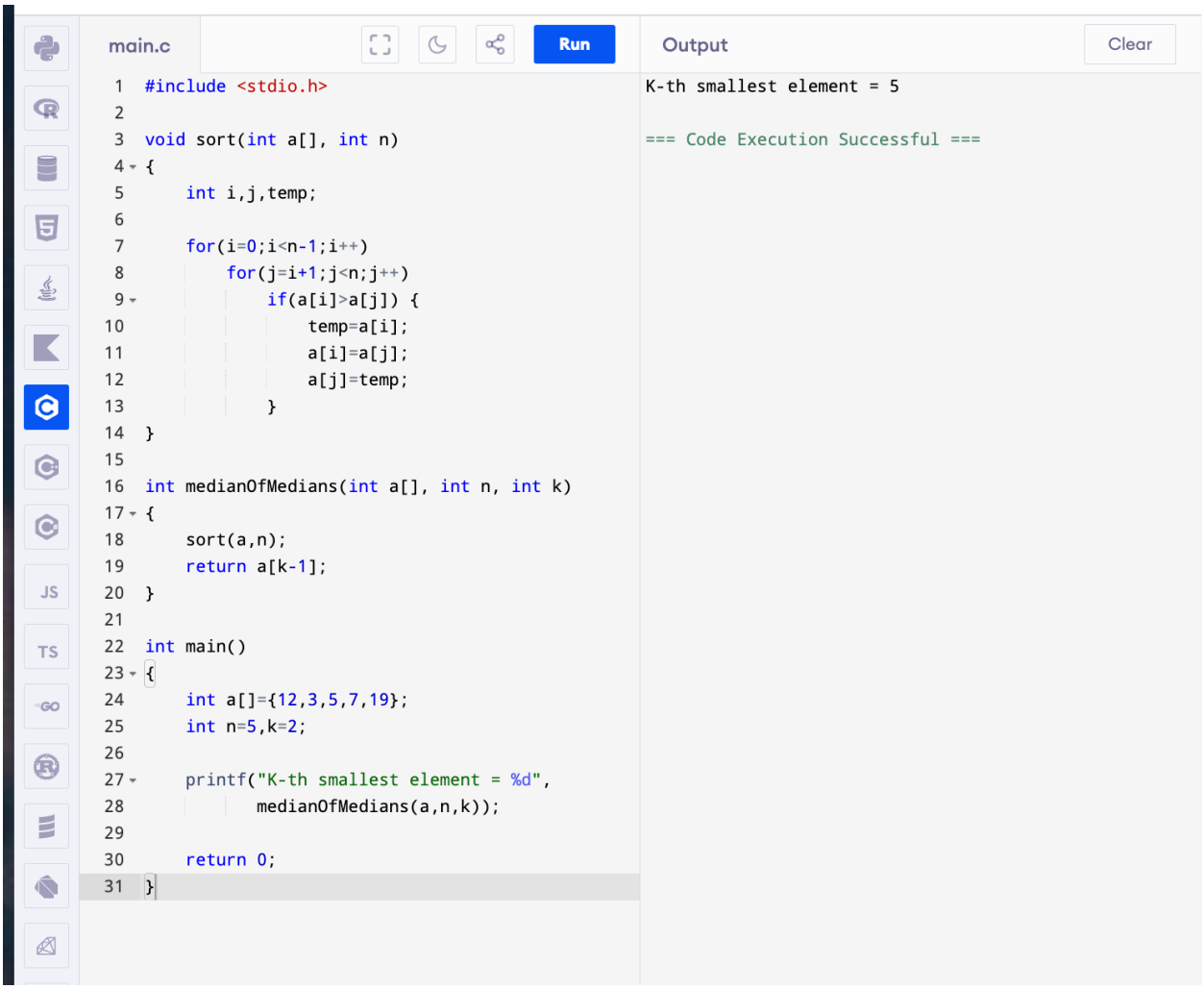
Clear

```
Closest points: [0,1] [-2,2]

=== Code Execution Successful ===
```

10. Given four lists A, B, C, D of integer values,Write a program to compute how many tuples n(i, j, k, l) there are such that A[i] + B[j] + C[k] + D[l] is zero.



11. To Implement the Median of Medians algorithm ensures that you handle the worst-case time complexity efficiently while finding the k-th smallest element in an unsorted array.



12. To Implement a function median_of_medians(arr, k) that takes an unsorted array arr and an integer k, and returns the k-th smallest element in the array.

main.c

Run

Output

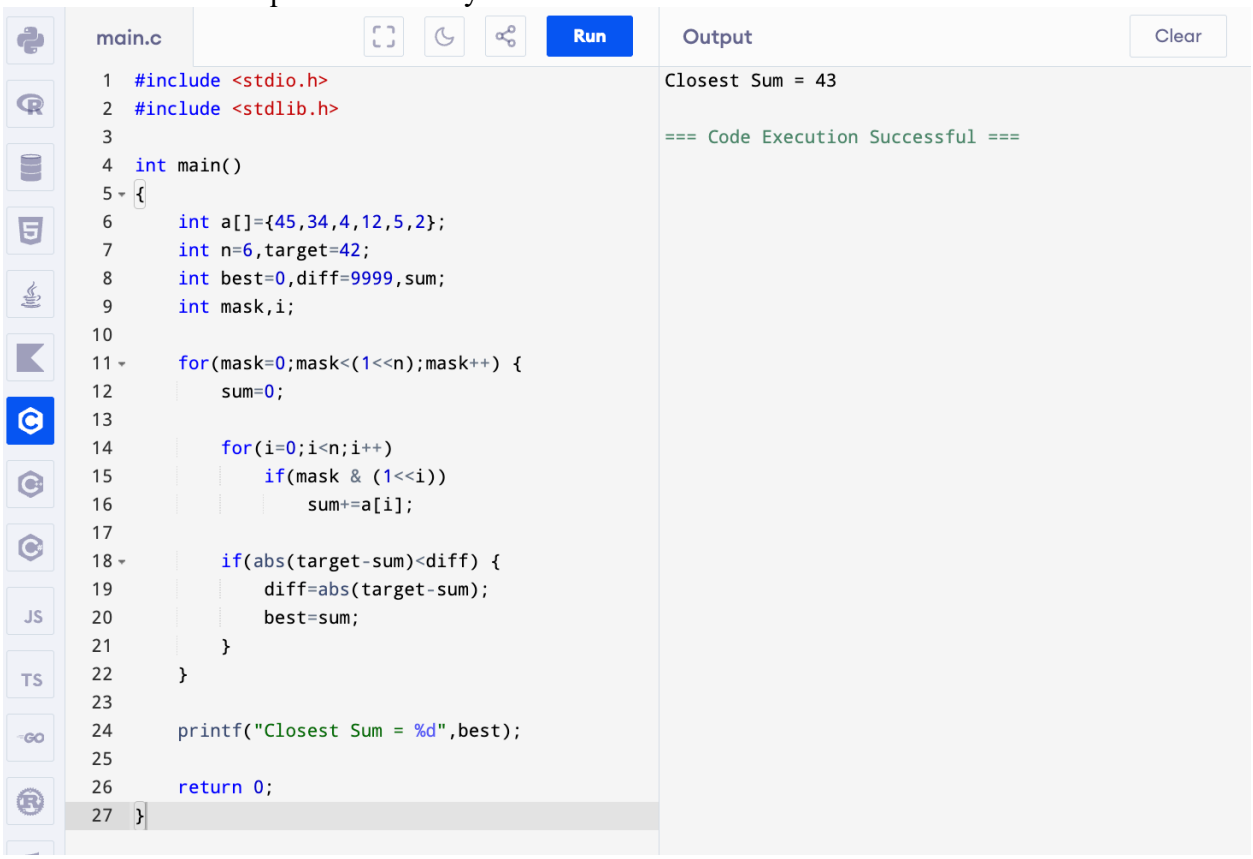
Clear

```c
#include <stdio.h>

void sort(int a[], int n)
{
    int i,j,temp;

    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]>a[j]) {
                temp=a[i];
                a[i]=a[j];
                a[j]=temp;
            }
}

int median_of_medians(int a[], int n, int k)
{
    sort(a,n);
    return a[k-1];
}

int main()
{
    int a[]={1,2,3,4,5,6,7,8,9,10};
    int b[]={23,17,31,44,55,21,20,18,19,27};

    printf("Test 1 = %d\n",median_of_medians(a
        ,10,6));
    printf("Test 2 = %d",median_of_medians(b,10
        ,5));

    return 0;
}
```

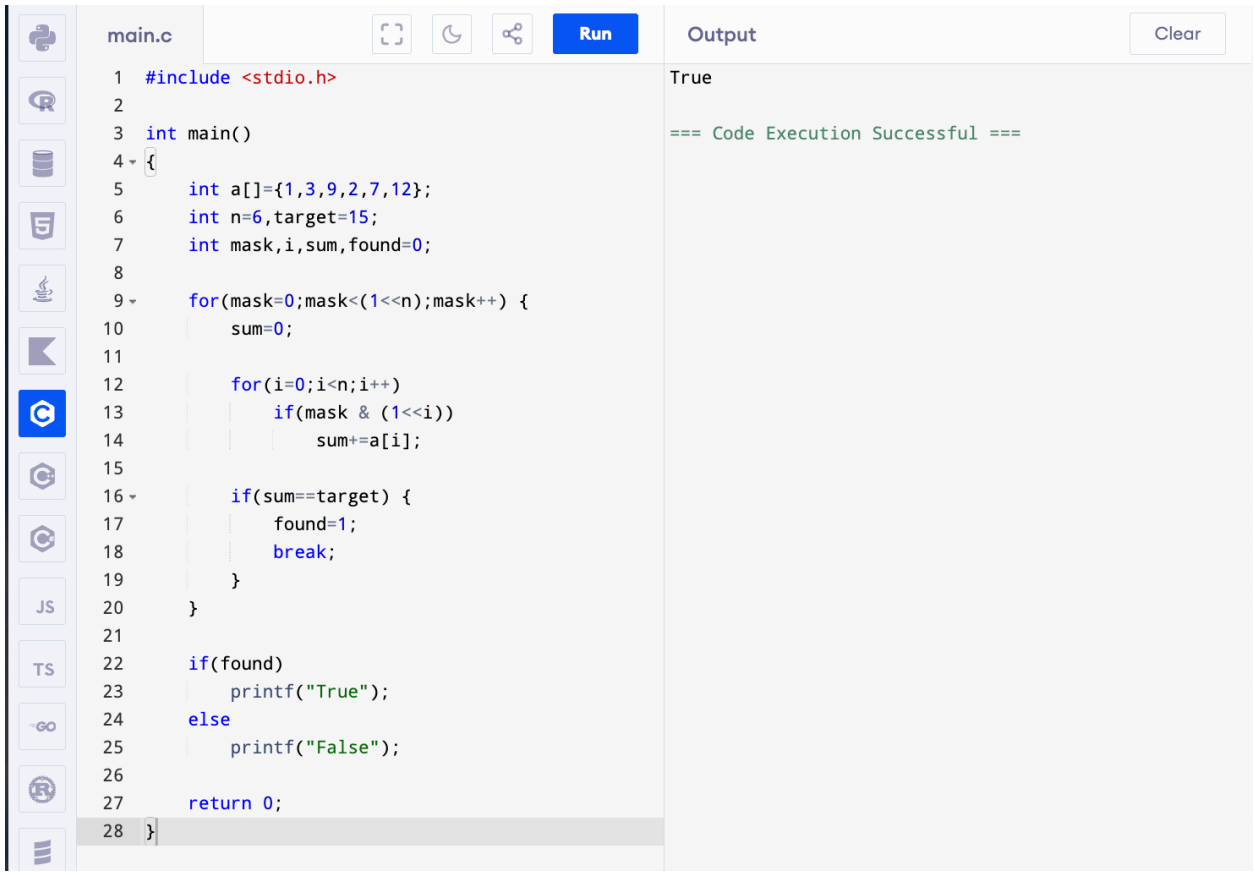
```
Test 1 = 6
Test 2 = 21

=== Code Execution Successful ===
```

13. Write a program to implement Meet in the Middle Technique. Given an array of integers and a target sum, find the subset whose sum is closest to the target. You will use the Meet

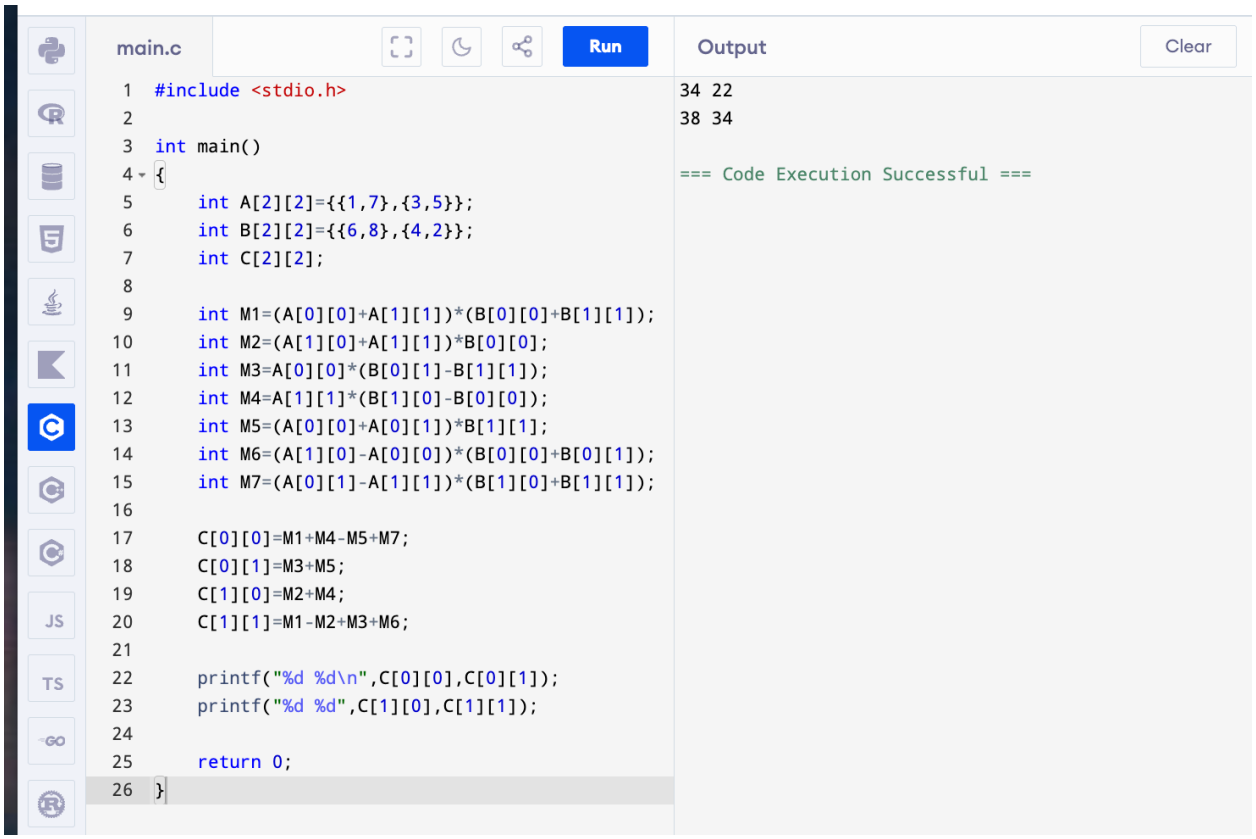
in the Middle technique to efficiently find this subset.



14. Write a program to implement Meet in the Middle Technique. Given a large array of integers and an exact sum E, determine if there is any subset that sums exactly to E. Utilize the Meet in the Middle technique to handle the potentially large size of the array. Return true if there is a subset that sums exactly to E, otherwise return false.



15 Given two 2×2 Matrices A and B



16 Given two integers X=1234 and Y=5678: Use the Karatsuba algorithm to compute the product Z=X x Y

main.c

Run

Output

Clear

```c
#include <stdio.h>

long long karatsuba(long long x, long long y)
{
    long long a,b,c,d,p=1;
    long long z0,z1,z2;

    if(x<10 || y<10)
        return x*y;

    while(x/p>=10)
        p*=10;

    a=x/p;
    b=x%p;
    c=y/p;
    d=y%p;

    z0=karatsuba(b,d);
    z2=karatsuba(a,c);
    z1=karatsuba(a+b,c+d)-z2-z0;

    return z2*p*p+z1*p+z0;
}

int main()
{
    long long x=1234,y=5678;

    printf("Z = %lld",karatsuba(x,y));

    return 0;
}
```

```
Z = 7006652

=== Code Execution Successful ===
```